

Java Annotations

Page No. :-

Date :- / /

Q

What is Annotation?

Start with @

- It is kind of adding META DATA to the java code
- Means, its usage is OPTIONAL.
- We can use this meta data information at runtime and can add certain logic in our code if wanted.
- How to read meta data information?
Ans using Reflection as discussed in previous video.
- Annotation can be applied at anywhere like classes, methods, interface, fields, parameters etc.

Ex: @Override → is an annotation

Types of Annotations

Pre-Defined Annotations

Custom Annotations /
User Defined

Used on Annotations

(. @ {our custom name?})

(called Meta Annotations)

Used on Java code

Eg: @Mohit

like classes, methods etc.

- @Target
- @Retention
- @Documented
- @Inherited
- @Repeatable
(Java 8 feature)

- @Deprecated
- @Override
- @SuppressWarnings
- @FunctionalInterface
- @SafeVarargs

Annotations used on Java code:

1. @Deprecated:

- Usage of Deprecated Class or Method or fields, shows you compile time WARNING.
- Deprecation means, no further improvement is happening on this and use new alternative method or field instead.
- Can be used over : Constructors, field, Local Variable, Method, Package, Type (class, interface, enum)

```
Ex: public class Mobile {
    @Deprecated
    public void dummyMethod() {
        // do something
    }
}
```

```
public class Main {
    psum() {
```

```
        Mobile mobile = new Mobile();
```

```
        mobile.dummyMethod();
    }
}
```

bulb like
warning
Shows
Compiler warning

2. @Override:

- During compile time, it will check that the method should be overridden.
- And throws compile time error, if it do not match with the parent method.
- Can be used over : METHODS

3. @SuppressWarnings:

- It will tell compiler to IGNORE any compile time WARNING.
- Use it safely, could lead to Run time exception if, any valid warning is IGNORED.
- Can be used over: Field, Method, Parameter, constructor, Local Variable, Type (class or interface or enum).

Ex: Class Mobile {

@Deprecated

void dummyMethod() {

//

}

}

Class Main {

② @SuppressWarnings("deprecation")

psvm() {

Mobile mobile = new Mobile();

mobile.dummyMethod();

}

}

① use then say class level

or

② shows method level
used here

@SuppressWarnings("all") // Ignore all warning

,

("unused") // method is unused but defined only.

4. @FunctionalInterface:

- Restrict interface to have only 1 abstract method.
- Throws compilation error, if more than 1 abstract method found.
- Can be used over: Type (class or interface or enum).

5. @SafeVarargs:

- Used to suppress "Heap pollution warning"
- Used over methods and constructors which has Variable Arguments as parameter.
- Method should be either static or final (i.e. methods which can not be overridden)
- In Java9, we can also use it on private methods too.

What is Heap Pollution?

→ Object of one type (Example String), storing the reference of another type Object (Ex Integer)

A obj $\Rightarrow$ B obj

In A type obj we are trying to store B type obj called as heap pollution

Ex: public class Log {

@SafeVarargs

public static void printLogValue(List<Integer>... list)

Object[] object = list // heap pollution caused

List<String> strList = new ArrayList<>();

strList.add("Mohit");

Object[] object = strList;

?

?

Annotations used over Another Annotations (META-ANNOTATIONS):

1. @Target:

- This meta-annotation will restrict, where to use the annotation.

Either at method level or constructor or field etc.

- @Interface is used to define an annotation

```
@Target(ElementType.METHOD)
public @Interface Override {
}
```

Override is used at method level

META-ANNOTATION

↓
Apply another
annotation over
one annotation
↓
one annotation

```
@Target(ElementType.CONSTRUCTOR,
          ElementType.METHOD)
public @Interface SafeVarargs {
}
```

Element Types are:

FIELD , PARAMETER , LOCAL-VARIABLE
METHOD , CONSTRUCTOR , PACKAGE etc.

2. @Retention:

- This meta-annotation tells, how Annotation will be stored in java.

- RetentionPolicy.SOURCE : Annotations will be discarded by the compiler itself and it will not be recorded in .class file.
- RetentionPolicy.CLASS : Annotations will be recorded in .class file but will be ignore by JVM at run time. → Can't use reflection (happens at runtime).

Page No:
 Date:
 → we will be able to apply reflection here

- RetentionPolicy.RUNTIME: Annotations will be recorded in .class file + available during run time. usage of reflection can be done.

Ex1. @Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Override {}

```
public class Eagle implements Bird {  
    @Override  
    public void fly() {  
        // code  
    }  
}
```

↓ when Eagle class compile its
• class file looks like

```
public class Eagle implements Bird {  
    public Eagle() {}  
    public void fly() {  
        // code  
    }  
}
```

→ we will see @Override is missing as we apply RetentionPolicy at Source means annotation not available in .class

3. @Documented:

- By default, Annotations are ignored when Java Documentation is generated. Eg @Override miss in
- With this meta-annotation even Annotations will come in Java Docs (ex @SafeVarargs has @Documented annotation by default.)

@Inherited

- By default, Annotations applied on parent class and not available to child classes.
- But it is after this meta-annotations.
- This meta-annotation has no effect, if annotation is used other than a class.

Ex: @Inherited ←

```
@Target(ElementType.TYPE)
```

```
@Retention(RetentionPolicy.RUNTIME)
```

```
public @interface MyCustomAnnotationWithInherited {  
    // custom Annotation  
}
```

```
@MyCustomAnnotationWithInherited
```

```
public class ParentClass {  
}
```

→ Here no custom annotation mentioned but due to @Inherited child class can inherit all annotations of parent class

```
public class ChildClass extends ParentClass {  
}
```

```
public class Main {
```

```
    psvm()
```

```
        sout(new ChildClass().getClass().getAnnotation(  
            MyCustomAnnotationWithInherited.class));  
    }
```

```
}
```

Output : @MyCustomAnnotationWithInherited()

Note: if we remove @Inherited and run the same program
output: null

@Repeatable :

- Allow us to use the same annotation more than 1 at same place.
- we cannot do this before JAVA8.

Ex1. `@Deprecated` ~~X~~ Not possible
`@Deprecated`
`public class Eagle {`
 `public void fly() { }`

We need to use @Repeatable Meta-annotation

Ex2: `@Repeatable({Categories.class})` ← This is a container/array of storing category
`@interface Category {`
 `String name();` // Step 1
`}`

`@Retention(RetentionPolicy.RUNTIME)`
`@interface Categories {`
 `Category[] value();` // Step 2
`}`

// Now we can use multiple repetition of same annotation

```
@Category(name = "Bird")
@Category(name = "Living Things")
@Category(name = "Carnivorous")
public class Eagle {
    public void fly() { }
```


public class Main {

public static void main (String args[]) {

Category[] categoryAnnotationArray = new Category[1];

categoryAnnotationArray[0] = new Category("Bird", "LivingThing", "Carnivorous");

for (Category annotation : categoryAnnotationArray) {

System.out.println(annotation.name());

}

}

}

Output: Bird

LivingThing

Carnivorous

User Defined or Custom Annotations:

- We can create our own ANNOTATION using keyword "@interface"

1. Creating an Annotation with empty body:

```
public @interface MyCustomAnnotation {
```

```
}
```

2. Creating an Annotation with method (its more like field)

- No parameter, no body

- Return type is restricted to primitive, class, String, enums, annotations and array of these types.

// we restrict our annotation by using @Target, @Retention etc.

```
public @interface MyAnnotation {
```

```
String name();
```

```
int value() default "hello";
```

```
}
```



```
@MyAnnotation(name = "testing"), {value = "Hello"}  
public class Eagle {  
    public void fly() { }  
}
```

by default taken
if we donot pass
any value.